HW 4: Tutorial

AI for Health

Note on this document

Files for this assignment can be found at this public repository:

While the pdf is helpful. I imagine that running the .jupytr notebook or the .qmd file will be more interactive.

Explanations on setting up the environment are on the next slide and in the README.md

Using RAG to Augment a Small Model

RAG stands for Retrieval-Augmented Generation and is a way to augment AI models with topic-specific information.

In this tutorial, we will set up RAG on a small language model that can provide reasonable advice about Lyme's Disease on a local machine.

Outline

The goal is to make a capable model that can run locally and give reasonable medical advice.

We will download the model from Hugging Face using the `transformers` package.

We will gather cdc data using the `Crawl4AI` package.

We will process that data and save it in a local database using the `chromadb` package.

We will give the bot a "memory" by saving conversations as a json file.

Finally we will run a chat with the bot about Lyme's Disease.

Choosing a model

While there are many small models, I wanted a model that could run locally and didn't already know lots of information about Lyme's Disease.

I settled on Hugging Face's SmolLM2 family of models.

Hugging Face offers three SmolLM2 models: SmolLM2-135M, SmolLM2-360M, and SmolLM2-1.7B (from smallest to largest).

After some experimentation, the SmolLM2-1.7B model proved capable but too slow on local hardware.

SmolLM2-135M was a bit dumb, and although it gave responses quickly, it repeated itself and was often unhelpful even after RAG implementation.

Hugging Face's SmolLM2-360M model was both quick and gave reasonably coherent answers, and it didn't know much about Lyme Disease initially. This is the model I chose.

Step 1: Set up environment

For reproducibility, I'm including my environment specifications.

See README.md for setting up your environment in Conda. If you are familiar with Conda, run:

```
conda env create -f hw_4_AI_Health.yml
```

OR you can install: - Python 3.11 - transformers 5.18.0 - chromadb 1.5.9 - crawl4ai 0.9.4

Loading all packages

```
import chromadb
import json
import re
from transformers import pipeline
from chromadb.utils import embedding_functions
from crawl4ai import AsyncWebCrawler, CrawlerRunConfig
from crawl4ai.deep_crawling import BFSDeepCrawlStrategy
from crawl4ai.deep_crawling.filters import FilterChain, URLPat
```

Step 2: Load and save SmolLM2-360M-Instruct

We use `pipeline` to create the generator for the text.

The pipeline function has two arguments: 1. A task ('text_generation' in this case). 2. A model (we've pointed toward Hugging Face, and so will download this model).

No login is required for this model, but we do get a warning encouraging us to set up a token.

```
#generator = pipeline("text-generation", model="HuggingFaceTB/
#generator = pipeline("text-generation", model="HuggingFaceTB/
generator = pipeline("text-generation", model="HuggingFaceTB/S
```

```
Loading weights:   0%|          | 0/290 [00:00<?, ?it/s]
```

Step 3 (optional): Save the model to run locally

If not saved locally, the model will be loaded into memory. We use a well-named method from the `TextGenerationPipeline` object to save our model locally.

The model is so small that it's not challenging to save it locally. SmolLM2-360M takes up about 0.75 GB. For comparison:

- 0.25 GB for SmolLM2-135M

or

- 3.5 GB for SmolLM2-1.7B

```
generator.save_pretrained("./SmolLM2-360M-Instruct")
print("Model successfully saved locally!")
```

```
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]

Model successfully saved locally!
```

Note the files in our model

Our model consists of several files; the two most interesting are `tokenizer.json` and `model.safetensors`.

The tokenizer breaks up all the words into tokens used to create the model.

`model.safetensors` is the model itself. The format was created by Hugging Face to prevent spreading malicious code, hence "safe."

Step 3 (optional): Load saved model

Once the model is saved, it's not difficult to load it again with `pipeline()`. You simply point the model parameter at the local file instead of at HuggingFaceTB.

```
# Load the model directly from your local folder

generator = pipeline("text-generation",
    model="./SmolLM2-360M-Instruct")
```

Example prompt and output about Lyme's Disease

Temperature is a measure of the unpredictability of a model's outputs.

Temperature closer to 0 is more deterministic; temperature closer to 1 is more random.

When I prompt SmolLM2 with "Complete the sentence, Lyme Disease is" with a temperature of 0.1, it provides: "Complete this sentence, Lyme Disease is 100% preventable."

When I run the same code with a temperature of 0.9, it generates: "Complete this sentence, Lyme Disease is 40% more likely to be fatal in children than in adults…."

The first statement is correct; the second is not.

```
output = generator(
    "Complete this sentence, Lyme Disease is ",
    max_new_tokens=250,
    temperature = 0.1 )
print(output[0]['generated_text'])
```

Step 4: Save and load messages

Previous chats with the bot can be saved and loaded into memory so an ongoing conversation with the user can take place.

The following functions save and load chats into JSON files.
load_messages() save_messages()

(see the .jupytr notebook for the code)

Step 5: Add a memory to the model

Before RAG implementation, the conversation with the bot is okay—it covers lots of basic information. The temperature is set to 0.1.

The conversation falls apart when I ask: "What is the story of the discovery of Lyme's Disease?"

To which the model replies:

SmolLM2: "The story of Lyme's Disease was first reported in 1879 in Connecticut by Dr. George Hitchings."

This is not true. Since each conversation produces slightly different results, this particular conversation has been saved as chat_preRag_temp_0.1.json.

Run the chat

After running this code, the chatbot runs in the terminal; feel free to talk with it.

(see the .jupytr notebook for the code)

```
## Copilot helped me develop this code, using models from Clau

def lyme_chat():
    messages = load_messages()
    print("Chatbot Initialized! Type 'quit' to exit.\n")
    while True:
        # Get user input
        user_input = input("You: ")
        if user_input.lower() == 'quit':
            break

        # Append the user's new message to the memory bank
        messages.append({"role": "user", "content": user_input
```

Step 6: Scraping and tokenizing data

To implement RAG for SmolLM2, I had to gather data. Traditionally, I would have used R with the `Rvest` package, but Claude showed me a similar technology for Python: `Crawl4AI`, which specifically converts pages to markdown for preprocessing.

After the pages are scraped, the data will be loaded into the lyme_knowledge_db, which is what our model will reference when answering questions.

Using Crawl4AI

I scraped the CDC's website, specifically pages starting with `www.cdc.gov/lyme`.

These pages were converted to markdown by Crawl4AI and saved to a database called `lyme_knowledge_db` using `chromadb`.

Note: This script will ask you to install Playwright, so watch the terminal.

(see the .jupytr notebook for the code)

```
[INIT].... → Crawl4AI 0.9.4
[FETCH]... ↓ https://cdc.gov/lyme
[SCRAPE]..   https://cdc.gov/lyme
[COMPLETE]   https://cdc.gov/lyme
[FETCH]... ↓ https://www.cdc.gov/lyme/
[SCRAPE]..   https://www.cdc.gov/lyme/
[COMPLETE]   https://www.cdc.gov/lyme/
```

Step 7: Have a chat

It is now possible to have a chat where SmolLM2 will reference the data saved in lyme_knowledge_db.

Note: we load the database with `chromadb` and use the default sentence embedding model `all-MiniLM-L6-v2`.

Chromadb does the work of finding something similar to what the user asked and passing that to SmolLM2 via messages.

(see the .jupytr notebook for the code)

Further study

If you are interested (and have time), these topics can be used for further study:

- Examine different small models to see how they perform.
- Change the embedding model from "all-MiniLM-L6-v2".
- Attempt to make the model work for multiple medical topics.
- Make inference faster with the `accelerate` package.

Conclusion

In this tutorial, I showed how to download a Hugging Face model and augment it with data from a publicly facing website. This type of exercise can be used to create a chatbot for a website or, if trained on more data, to create a digital doctor knowledgeable about many different diseases.

Further reading

On SmolLM2 models

Understanding RAG

This project was completed with assistance from Claude, especially for the new packages: `crawl4ai` and `chromadb` which I knew nothing about before this assignment.